

Listes

Définition

- Une collection d'éléments séparés par des virgules, l'ensemble étant enfermé dans des crochets

```
>>> jour = ['lundi', 'mardi', 'mercredi', 1800, 20.357, 'jeudi', 'vendredi']  
>>> print(jour)  
['lundi', 'mardi', 'mercredi', 1800, 20.357, 'jeudi', 'vendredi']
```

- Les éléments individuels qui constituent une liste peuvent être de types variés
- On accède aux éléments d'une liste par son *index* dont la numération commence à partir du 0 (et non de 1)

```
>>> jour = ['lundi', 'mardi', 'mercredi', 1800, 20.357, 'jeudi', 'vendredi']  
>>> print(jour[2])  
mercredi  
>>> print(jour[4])  
20.357
```

Définition d'une liste – accès à ses éléments

```
>>> nombres = [5, 38, 10, 25]
>>> mots = ["jambon", "fromage", "confiture", "chocolat"]
>>> stuff = [5000, "Brigitte", 3.1416, ["Albert", "René", 1947]]
>>> print(nombres[2])
10
>>> print(nombres[1:3])
[38, 10]
>>> print(nombres[2:3])
[10]
>>> print(nombres[2:])
[10, 25]
>>> print(nombres[:2])
[5, 38]
>>> print(nombres[-1])
25
>>> print(nombres[-2])
10
```

Déterminer la longueur d'une liste : fonction len()

```
>>> print(len(jour))
7
```

- Le niveau d'indentation n'a pas d'importance :
l'interpréteur détecte la fin de l'instruction

```
couleurs = ['noir', 'brun', 'rouge',  
            'orange', 'jaune', 'vert',  
            'bleu', 'violet', 'gris', 'blanc']
```

Appartenance d'un élément à une séquence : l'instruction *in* utilisée seule

```
n = 5
premiers = [1, 2, 3, 5, 7, 11, 13, 17]
if n in premiers:
    print(n, "fait partie de notre liste de nombres premiers")
```

Opérations sur les listes

+ (concatenation)

```
>>> fruits = ['orange', 'citron']  
>>> legumes = ['poireau', 'oignon', 'tomate']  
>>> fruits + legumes  
['orange', 'citron', 'poireau', 'oignon', 'tomate']  
>>> fruits * 3  
['orange', 'citron', 'orange', 'citron', 'orange', 'citron']
```

* (multiplication)

```
>>> sept_zeros = [0]*7  
>>> sept_zeros  
[0, 0, 0, 0, 0, 0, 0]
```

Opérations sur les listes

- Changer des éléments d'une liste (contrairement aux chaînes de car. qui constituent des données non-modifiables)

```
>>> print(jour)
['lundi', 'mardi', 'mercredi', 1800, 20.357, 'jeudi', 'vendredi']
>>> jour[3] = jour[3] + 47
>>> print(jour)
['lundi', 'mardi', 'mercredi', 1847, 20.357, 'jeudi', 'vendredi']
```

- Remplacer des éléments d'une liste par d'autres

```
>>> jour[3] = 'Juillet'
>>> print(jour)
['lundi', 'mardi', 'mercredi', 'Juillet', 20.357, 'jeudi', 'vendredi']
```

- Supprimer d'une liste un élément : fonction *del()*

```
>>> del(jour[4])
>>> print(jour)
['lundi', 'mardi', 'mercredi', 'juillet', 'jeudi', 'vendredi']
```

Opérations sur les listes en tant qu'objet

```
>>> nombres = [17, 38, 10, 25, 72]
>>> nombres.sort()           # trier la liste
>>> nombres
[10, 17, 25, 38, 72]

>>> nombres.append(12)       # ajouter un élément à la fin
>>> nombres
[10, 17, 25, 38, 72, 12]

>>> nombres.reverse()        # inverser l'ordre des éléments
>>> nombres
[12, 72, 38, 25, 17, 10]

>>> nombres.index(17)         # retrouver l'index d'un élément
4

>>> nombres.remove(38)        # enlever (effacer) un élément
>>> nombres
[12, 72, 25, 17, 10]
```


Insérer des objets dans une liste

```
liste=[1,2,3]
```

```
liste.append(7)           #on ajoute 7 à la fin de la liste
```

- On passe en paramètre de la méthode *append* l'objet que l'on souhaite ajouter à la fin de la liste

```
liste.insert(2, 'c')      #on insère 'c' à l'indice 2
```

```
print (liste)
```

- Quand on demande d'insérer c à l'indice 2, la méthode va décaler les objets d'indice supérieur ou égal à 2.

```
liste1=[1,2,3]
```

```
liste2=[6,7,8]
```

```
liste1.extend(liste2)     #on insère liste2 à la fin de la liste1
```

```
print (liste1)
```

Supprimer un élément de la liste

- Méthode: *remove*

```
>>> nombres.remove(38)                # enlever (effacer) un élément
>>> nombres
[12, 72, 25, 17, 10]
```

⇒ La méthode *remove* prend en paramètre l'élément à supprimer

⇒ Elle parcourt la liste et en retire l'élément qu'on lui passe en paramètre

- Fonction : *del*

```
>>> del nombres[2]
>>> nombres
[12, 72, 17, 10]
>>> del nombres[1:3]
>>> nombres
[12, 10]
```

⇒ *del* sert plus généralement à supprimer non seulement les éléments d'une séquence mais aussi des variables

⇒ *del* n'est pas une méthode mais une fonctionnalité de Python

⇒ Elle prend en paramètre un index de l'élément à supprimer

Insertion d'un ou plusieurs éléments n'importe où dans une liste

```
>>> mots = ['jambon', 'fromage', 'confiture', 'chocolat']
>>> mots[2:2] = ["miel"]
>>> mots
['jambon', 'fromage', 'miel', 'confiture', 'chocolat']

>>> mots[5:5] = ['saucisson', 'ketchup']
>>> mots
['jambon', 'fromage', 'miel', 'confiture', 'chocolat', 'saucisson', 'ketchup']
```

Suppression / remplacement d'éléments

```
>>> mots[2:5] = [] # [] désigne une liste vide
>>> mots
['jambon', 'fromage', 'saucisson', 'ketchup']

>>> mots[1:3] = ['salade']
>>> mots
['jambon', 'salade', 'ketchup']

>>> mots[1:] = ['mayonnaise', 'poulet', 'tomate']
>>> mots
['jambon', 'mayonnaise', 'poulet', 'tomate']
```

La fonction *range()*

- renvoie une séquence d'entiers qu'on peut utiliser directement ou convertir en une liste avec la fonction **list()**

```
>>> list(range(10))
```

```
list(range(5, 13))
```

```
list(range(3, 16, 3))
```

```
list(range(10, -10, -3))
```

```
mon_objet_range = range(0, 4) # 0 1 2 3

indexation = mon_objet_range[2]

print(indexation) # 2
```

Dans cet exemple, la variable `indexation` vaudra 2 car `mon_objet_range` est similaire à une liste `[0, 1, 2, 3]` et l'élément en position 2 de cette liste est le nombre 2.

On peut aussi découper l'objet range :

```
mon_objet_range = range(0, 4) # 0 1 2 3

print(mon_objet_range[:2]) # 0 1
print(mon_objet_range[2:]) # 2 3
print(mon_objet_range[1:2:3]) # 1
```

Parcours d'une liste

```
>>> prov = ['La', 'raison', 'du', 'plus', 'fort', 'est', 'toujours', 'la', 'meilleure']
>>> for mot in prov:
...     print(mot, end = ' ')
...
La raison du plus fort est toujours la meilleure
```

```
>>> for n in range(10, 18, 3):
...     print(n, n**2, n**3)
...
10 100 1000
13 169 2197
16 256 4096
```

Utilisation de la séquence générée par `range()` pour accéder aux éléments d'une liste :

```
fable = ['Maître', 'Corbeau', 'sur', 'un', 'arbre', 'perché']
for index in range(len(fable)):
    print(index, fable[index])
```

L'exécution de ce script donne le résultat :

```
0 Maître
1 Corbeau
2 sur
3 un
4 arbre
5 perché
```

Trier les listes

- Fonction ***sorted*** : permet de parcourir les éléments d'une liste de façon ordonnée => renvoie une nouvelle liste triée :

```
>>>sorted([5, 2, 3, 1, 4])    li = [4, 5, 3, -6, 7, 9]
[1, 2, 3, 4, 5]              for n in sorted(li):
                               print(n)
                               print(li) # la liste li n'est pas triée
```

- Méthode ***list.sort()*** :

- modifie la liste elle-même (et renvoie None pour éviter les confusions).
- est moins pratique que la fonction `sorted()`, mais si vous n'avez pas besoin de la liste originale, cette technique est légèrement plus efficace.

```
>>> a = [5, 2, 3, 1, 4]      li = [4, 5, 3, -6, 7, 9]
>>> a.sort()                li.sort() # la liste est triée
>>> a                        for n in li:
[1, 2, 3, 4, 5]              print(n)
```

- `list.sort()` est seulement définie pour les listes, la fonction `sorted()` accepte n'importe quel itérable.

```
>>> sorted({1: 'D', 2: 'B', 3: 'B', 4: 'E', 5: 'A'})    #il s'agit de dictionnaire voir plus loin
[1, 2, 3, 4, 5]
```

Fonctions

- La fonction *enumerate* permet d'éviter l'emploi de la fonction *range* lorsqu'on souhaite parcourir une liste alors que **l'indice et l'élément** sont nécessaires.

```
li = [4, 5, 3, -6, 7, 9]
```

```
for i in range(0, len(li)):
    print(i, li[i])
```

```
for i, v in enumerate(li):
    print(i, v)
```


Parcours de la liste

```
>>>liste=[1,2,3]
>>>for ele in liste :
    print (ele)
```

Inconvénient : *for* parcourt la liste en capturant les éléments dans une variable sans qu'on puisse savoir où ils sont dans la liste

```
>>>for ele in enumerate(liste) :
    print (ele) #résultat des tuples
```

Quand on parcourt chaque élément de l'objet renvoyé par *enumerate*, on voit des tuples qui contiennent deux éléments : l'indice, puis l'objet se trouvant à cet indice

```
>>>for i, ele in enumerate (liste) :
    print(i, ele)
```

Enumerate prend en paramètre une liste et renvoie un objet qui peut être associé à une liste contenant deux valeurs par élément : l'indice et l'élément de la liste parcouru

Liste compactes : Parcours de la liste

```
liste=[1,2,3]
```

```
print([nb * nb for nb in liste])
```

- la valeur de retour (ici mettre au carré les nombres), on pouvait écrire `nb**2`
- La syntaxe d'une boucle *for*, voilà d'où vient la variable `nb`

Python va parcourir la liste d'origine et mettre chaque élément de la liste au carré. Il renvoie ensuite le **résultat obtenu**, sous la forme d'**une liste** qui est de la même longueur que celle d'origine.

```
liste=[1,2,3]
```

```
for nb in liste:
```

```
    print(nb * nb) #le résultat n'est pas une liste
```

Liste compactes

```
l = [ ]  
for i in range (10) :  
    l.append( i*2+1)  
l
```

=> c'est quoi le résultat ?

On peut écrire :

```
l = [ i*2+1 for i in range(10) ]  
l
```

Listes compactes : Filtrage avec un branchement conditionnel

```
>>>liste=[1,2,3]
```

```
>>>[nb for nb in liste if nb %2 ==0]
```

```
...
```

On ajoute la condition qui va déterminer quelles valeurs seront transférées dans la nouvelle liste :
ici les valeurs paires

Exercices sur les listes compactes

```
l = [ i*2 for i in range(0,10) ]  
l # qu'affiche l ?
```



```
l = [ i*2 for i in range(0,10) if i%2==0 ]  
l # qu'affiche l ?
```



Exercices

On veut écrire quelques instructions pour trouver la position du nombre `x = 7` dans la liste `l`.
Il faut compléter le programme suivant en utilisant une boucle et un test.

```
l = [ 3, 6, 2 , 7, 9 ]  
x = 7  
  
# .....  
  
print ( position )
```

Il n'y a pas d'autres moyens que de passer tous les éléments en revue et de conserver la position du premier élément dans la liste qui vérifie le critère de recherche.

```
l = [ 3, 6, 2 , 7, 9 ]  
x = 7  
  
for i,v in enumerate(l) :  
    if v == x :  
        position = i  
  
print ( position )
```

Travail à faire

Soient les listes suivantes :

1.

```
t1 = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
t2 = ['Janvier', 'Février', 'Mars', 'Avril', 'Mai', 'Juin',
      'Juillet', 'Août', 'Septembre', 'Octobre', 'Novembre', 'Décembre']
```

Écrivez un petit programme qui crée une nouvelle liste `t3`. Celle-ci devra contenir tous les éléments des deux listes en les alternant, de telle manière que chaque nom de mois soit suivi du nombre de jours correspondant :

```
['Janvier', 31, 'Février', 28, 'Mars', 31, etc...].
```

2. Rechercher le plus grand élément dans une liste de nombres donnée.

Input : `[13, 79, 1, 30, 456, 16]`

Output : **Le plus grand élément de la liste a la valeur de 456.**

3. A partir d'une liste de nombres donnée, créer deux nouvelles listes : avec les nombres divisibles par 4 et les autres.

Input : `[13, 79, 1, 30, 456, 16]`

Output :

liste des nombres divisibles par 4 : [456, 16]

liste des nombres non divisible par 4 : [13, 79, 1, 30]